

## Debacu Evaluation360 — Modelo Base de Importación CSV Unificado (v1)

### 1. Objetivo

Definir un formato estándar de CSV para hoteles piloto que permita:

- Screening automático de huéspedes por lote.
- Alimentar los módulos de Revenue Intelligence.
- Construir histórico analítico (pickup, comparativas YoY, dashboard).
- Evitar múltiples formatos por PMS durante la fase piloto.

El hotel exporta UN SOLO CSV desde su PMS o Excel y lo sube a Debacu cuando quiera.

### 2. Filosofía del modelo

- Un único archivo CSV.
- Una fila = una reserva / estancia.
- El mismo archivo sirve para:
  - Screening automático
  - Análisis de revenue
  - Pickup avanzado
  - Comparativas históricas

Debacu no obliga a una frecuencia fija de carga.

El hotel puede subir el CSV:

- diariamente
- semanalmente
- bajo demanda

Debacu procesa el lote completo y detecta cambios respecto a consultas anteriores.

### 3. Campos obligatorios (mínimo funcional)

reservation\_id

booking\_date

checkin\_date

checkout\_date

status

gross\_revenue

rooms

Descripción:

reservation\_id → identificador único de la reserva

booking\_date → fecha en la que se realizó la reserva

checkin\_date → fecha de entrada

checkout\_date → fecha de salida

status → confirmed / cancelled / no\_show / checked\_out

gross\_revenue → importe bruto de la reserva

rooms → número de habitaciones

### 4. Campos recomendados (Revenue Intelligence)

channel

segment

commission\_amount

net\_revenue

currency

adults

children

room\_type

rate\_plan

Estos campos permiten construir:

- mix por canal
- análisis por segmento
- revenue neto vs bruto
- ADR
- comparativas de canal

## 5. Campos para identificación de huésped (screening)

first\_name

last\_name

document

email

phone

country

Estos campos se utilizan para:

- detectar huéspedes problemáticos
- cruzar incidencias entre hoteles
- generar alertas de riesgo

Los datos se almacenan cifrados o hasheados.

## 6. Campos opcionales

cancelled\_at

no\_show

company

agency

market\_code

source

city

No son necesarios para la fase piloto pero pueden enriquecer el análisis.

## 7. Reglas de validación

Fechas:

Formato YYYY-MM-DD

Números:

Decimal con punto

Ejemplo: 1250.50

Coherencias:

checkout\_date >= checkin\_date

booking\_date <= checkin\_date

Revenue:

Debe ser >= 0

reservation\_id:

Debe ser único.

## 8. Flujo de importación en Debacu

1. El hotel descarga la plantilla CSV oficial Debacu.
2. Exporta datos desde su PMS o Excel.
3. Ajusta el formato si es necesario.
4. Sube el CSV a Debacu.

Debacu ejecuta:

- validación de estructura
- validación de datos
- preview del archivo
- screening automático
- ingestión analítica.

## 9. Uso del mismo CSV para múltiples funciones

Screening:

Detecta huéspedes con incidencias previas.

Revenue Intelligence:

Construye métricas como:

ADR

RevPAR

Ocupación

Revenue bruto/neto

Mix de canales

Pickup avanzado:

Calcula ritmo de reservas comparado con histórico.

## 10. Histórico recomendado

Para análisis avanzado se recomienda cargar:

- histórico de 24 meses
- reservas actuales
- reservas futuras

Esto permite:

- comparativas YoY
- análisis de estacionalidad
- modelos predictivos futuros.

## 11. Ejemplo de filas CSV

reservation\_id,booking\_date,checkin\_date,checkout\_date,status,channel,gross\_revenue,rooms,first\_name,last\_name,document,email,phone,country

RES001,2026-01-15,2026-03-10,2026-03-12,confirmed,BOOKING,450.00,1,Juan,Perez,12345678A,juan@email.com,600000000,ES

RES002,2026-01-18,2026-03-11,2026-03-14,confirmed,DIRECT\_WEB,780.00,1,Ana,Gomez,87654321B,ana@email.com,611111111,ES

RES003,2026-01-20,2026-03-15,2026-03-18,cancelled,EXPEDIA,520.00,1,Luis,Martinez,,luis@email.com,622222222,FR

## 12. Evolución futura del modelo

El modelo CSV v1 está pensado para pilotos.

En versiones posteriores se podrán añadir:

- integraciones PMS
- importaciones automáticas
- normalización avanzada de canales
- enriquecimiento de datos
- análisis con agentes IA.

revisado el documento base y encaja con la dirección correcta: un único CSV, una fila por reserva/estancia, y uso simultáneo para screening, revenue y pickup. Además, tu v1 ya marca como mínimos `reservation_id`, `booking_date`, `checkin_date`, `checkout_date`, `status`, `gross_revenue` y `rooms`, junto con un bloque de identificación y otro de revenue.

debacu\_csv\_model\_v1\_base

La realidad práctica, viendo también el CSV de prueba que estás usando ahora, es esta: **el CSV actual sirve para screening de laboratorio, pero no sirve como CSV unificado real**. Solo trae fechas y datos de identidad, no trae `reservation_id`, `booking_date`, `status`, `gross_revenue`, `rooms`, `channel`, `segment`, ni nada útil para Revenue Intelligence. Además, tiene `checkout_date` vacíos en varias filas. Eso para screening todavía puede pasar; para revenue y pickup, no.

Voy al grano.

## 1) Diagnóstico real

Tu documento v1 es válido como base de producto piloto, pero se queda corto para explotación SaaS real. Define bien la idea, pero todavía no cierra tres cosas críticas: clave natural de deduplicación, histórico de snapshots para pickup, y separación clara entre estado actual de reserva y trazabilidad histórica.

debacu\_csv\_model\_v1\_base

La decisión correcta es esta:

**Debacu mantiene “una fila = una reserva/estancia” en el CSV**, porque pedir al hotel un CSV por noche o por habitación-noche es inviable.

Luego, **Debacu explota internamente esa reserva en noches** para occupancy, ADR, RevPAR, price calendar y pickup.

Ese enfoque es realista. Pedir más al hotel no va a escalar.

---

## 2) CSV Debacu oficial definitivo

### Decisión de producto

Haz un único formato canónico, con orden fijo de columnas, y permite luego perfiles de mapeo por PMS.

El hotel no tiene que aprender 4 formatos. Debacu adapta.

### Cabecera oficial recomendada

`property_code`, `reservation_id`, `reservation_line_id`, `booking_date`, `checkin_date`, `checkout_date`, `reservation_status`, `currency`, `gross_revenue`, `commission_am`

ount, net\_revenue, rooms, adults, children, channel, segment, room\_type, rate\_plan, first\_name, last\_name, document, email, phone, country, cancelled\_at, company, agency, market\_code, source\_system

### Obligatorias

Estas son las que yo haría obligatorias para aceptar un **CSV unificado real**:

- property\_code
- reservation\_id
- booking\_date
- checkin\_date
- checkout\_date
- reservation\_status
- currency
- gross\_revenue
- rooms

### Recomendadas fuertes

Estas no deberían bloquear el piloto, pero sin ellas el módulo revenue queda bastante más flojo:

- reservation\_line\_id
- commission\_amount
- net\_revenue
- adults
- children
- channel
- segment
- room\_type
- rate\_plan

### Recomendadas para screening

Aquí no haría “todo o nada”. Haría una regla por calidad de identidad:

- first\_name
- last\_name
- document



- email
- phone
- country

#### Opcionales

- cancelled\_at
  - company
  - agency
  - market\_code
  - source\_system
- 

### 3) Regla importante: identidad mínima para screening

No pondría todos los datos personales como obligatorios duros. Eso te rompería imports reales.

La regla sensata es:

Cada fila debe traer **al menos uno** de estos identificadores fuertes:

- document, o
- email, o
- phone

Y además, recomendable:

- first\_name y last\_name

Si una fila no trae ninguno de esos tres identificadores fuertes, la importas para revenue si el resto está bien, pero **esa fila queda con screening no ejecutable** o con `identity_strength = LOW`.

Eso es mucho más realista que rechazar el fichero entero.

---

### 4) Valores y validaciones

#### Fechas

Formato único:

- YYYY-MM-DD

## Coherencias

- `booking_date <= checkin_date`
- `checkout_date > checkin_date`
- Si `reservation_status = CANCELLED`, `cancelled_at` debería venir informado
- Si `reservation_status = NO_SHOW`, `checkout_date` sigue existiendo; no se deja nulo

## Enum canónico para estado

No uses textos libres del PMS internamente. Normaliza a:

- `CONFIRMED`
- `IN_HOUSE`
- `CHECKED_OUT`
- `CANCELLED`
- `NO_SHOW`

Luego cada PMS se mapea a esos valores.

## Importes

- `gross_revenue >= 0`
- `commission_amount >= 0`
- `commission_amount <= gross_revenue`
- `net_revenue >= 0`

Regla práctica:

- si viene `net_revenue`, se respeta
- si no viene y sí viene `commission_amount`, entonces `net_revenue = gross_revenue - commission_amount`
- si no viene ninguno, `revenue neto` no se calcula todavía

## Moneda

- `currency` en ISO 4217: EUR, USD, etc.

## Habitaciones

- `rooms entero >= 1`

## Duplicados dentro del mismo archivo

Aquí no sería flexible:

- si se repite la misma clave natural en el mismo fichero, **rechazo la fila duplicada** y aviso
  - no conviene “quedarse con la última” sin control, porque eso en hoteles genera errores silenciosos
- 

## 5) Clave natural y deduplicación

Tu idea de usar `org_id + reservation_id` se queda corta.

No sirve bien cuando:

- una reserva tiene varias líneas
- una reserva cambia de room type
- una reserva multi-room viene desglosada por PMS
- una misma reserva se reimporta muchas veces

### Clave natural recomendada de reserva

La base debería ser:

- `org_id`
- `property_code`
- `reservation_id`
- `coalesce(reservation_line_id, '1')`

Con eso generas un `reservation_key` hash estable.

### Para screening

No usaría `org_id + identity_key + checkin_date` como clave principal.

Eso puede colisionar con:

- el mismo huésped con dos reservas
- reservas partidas
- cambios de estancia

Mejor:

- `org_id`
- `property_code`

- reservation\_key
- identity\_key

### identity\_key

Se genera a partir del mejor identificador disponible, normalizado:

Prioridad:

1. documento
2. email
3. teléfono
4. nombre + apellido + país como fallback débil

Y guardas también:

- identity\_strength: HIGH, MEDIUM, LOW
- 

## 6) Estrategia de ingestión correcta

No metas directamente el CSV en tablas finales. Hazlo por capas.

### Capa 1: batch

Tabla:

- debacu\_eval\_import\_batches

Guarda:

- id
- org\_id
- source\_file\_name
- source\_file\_sha256
- import\_profile\_id
- status
- rows\_total
- rows\_ok
- rows\_error
- imported\_at

## Capa 2: raw normalizado

Tabla:

- `debacu_eval_import_rows`

Guarda la fila ya parseada, con:

- `batch_id`
- `row_number`
- `raw_payload`
- `normalized_payload`
- `validation_errors`
- `canonical_reservation_key`

Esto te permite auditar sin perder el original.

## Capa 3: snapshot histórico

Tabla:

- `debacu_eval_reservation_snapshots`

Esta es la clave para pickup.

Cada import guarda el estado observado de la reserva en ese momento:

- `batch`
- `reservation_key`
- `status`
- `revenue`
- `checkin/checkout`
- `channel`
- `segment`
- `etc.`

Esto es append-only. No se pisa.

## Capa 4: estado actual

Tabla:

- `debacu_eval_reservations`

Aquí haces upsert por `reservation_key`.

Esta tabla representa la última verdad operativa conocida de cada reserva.

#### Capa 5: noches derivadas

Tabla:

- debacu\_eval\_stay\_nights

Explota cada estancia en noches:

- reservation\_key
- stay\_date
- room\_nights
- allocated\_gross\_revenue
- allocated\_net\_revenue
- channel
- segment
- room\_type

Sin esto no tendrás occupancy, ADR diario, RevPAR ni overlay serio sobre calendarios.

---

### 7) Modelo de tablas que sí tiene sentido

Yo dejaría algo así:

#### Ingesta

- debacu\_eval\_import\_batches
- debacu\_eval\_import\_rows
- debacu\_eval\_import\_profiles

#### Negocio actual

- debacu\_eval\_reservations
- debacu\_eval\_reservation\_identities
- debacu\_eval\_stay\_nights

#### Histórico y pickup

- debacu\_eval\_reservation\_snapshots
  - debacu\_eval\_pickup\_snapshots\_daily
- o bien derivarlo desde snapshots si prefieres no materializar al principio

## Screening

- screening\_runs
- screening\_results
- screening\_alerts

## Agregados para UI

- debacu\_eval\_revenue\_daily
- debacu\_eval\_revenue\_monthly
- debacu\_eval\_channel\_mix\_daily

Si ya tienes debacu\_eval\_guest\_stays, puedes reutilizarlo, pero yo no mezclaría en la misma tabla:

- PII/identidad
- estado de reserva
- snapshot histórico
- métricas diarias

Eso termina siendo un monstruo inmantenible.

---

## 8) Cómo alimentar Revenue Intelligence de verdad

Con el CSV definitivo, puedes calcular bien:

Sí

- ADR
- lead time
- room nights
- cancelaciones
- no-shows
- mix de canal
- mix de segmento
- revenue bruto/neto
- pickup entre imports
- comparativa mensual

- comparativa YoY
- estacionalidad básica

#### Sí, pero con derivación interna

- ocupación diaria
- RevPAR
- overlay en price calendar

Para esto necesitas:

- rooms\_count por propiedad en Debacu
- explosión en stay\_nights

#### No de forma seria solo con este CSV

- competitor pricing
- market demand externa
- paridad OTA
- price positioning vs comp set

Eso vendrá por integración o scraping, no del CSV.

---

### 9) Regla crítica para pickup

Si quieres pickup serio, **no basta con guardar el estado actual.**

Tienes que guardar qué reservas existían en cada carga.

Ejemplo:

- el hotel sube hoy un CSV con agosto
- dentro de 7 días sube otro
- Debacu debe poder decir: “para llegadas del 15 de agosto, hoy hay 28 room nights y hace 7 días había 21”

Eso solo sale de:

- reservation\_snapshots
- o agregados diarios por batch

Si sobrescribes la reserva y no guardas snapshots, pickup muerto.

---



## 10) Qué haría con tu CSV actual de pruebas

Tu CSV actual no lo tirarías. Lo usarías como:

### Perfil 1: screening-lite

Admite:

- checkin\_date
- checkout\_date
- first\_name
- last\_name
- document
- email
- phone
- country

Sirve para validar el pipeline de screening.

### Perfil 2: debacu-unified-v1

Exige el formato unificado definitivo.

Así no rompes lo que ya funciona, pero dejas claro que el CSV actual **no es el modelo oficial final**.

Eso es importante: no mezclar pruebas de screening con diseño SaaS definitivo.

---

## 11) Decisión de producto que te recomiendo

Mi recomendación, sin adornos:

1. **Mantén un único CSV oficial canónico.**
2. **Acepta perfiles de mapeo por PMS.**
3. **Conserva un modo screening-lite temporal** para no frenar pilotos.
4. **El unificado definitivo debe exigir bloque mínimo de reserva + revenue**, no solo identidad.
5. **Guarda snapshots append-only** desde el primer día.
6. **Deriva noches internamente**; no se lo pidas al hotel.
7. **Separa tablas de raw, current, snapshots e identity.**

Ese es el diseño realista y escalable.

---

## 12) Qué cambiaría respecto a tu documento v1

Tu v1 dice lo correcto a nivel conceptual, pero yo le haría estos cambios estructurales: un `property_code` claro, una `reservation_line_id` para evitar colisiones, identidad mínima por calidad en vez de obligatoriedad absoluta, y persistencia de snapshots históricos además del estado actual. Sin eso, el CSV sirve para piloto, pero no para un SaaS tipo Lighthouse/Duetto alimentado por importación.

`debacu_csv_model_v1_base`

## 13) Conclusión clara

El camino bueno es este:

- **CSV oficial único**
- **pipeline por capas**
- **snapshot histórico**
- **tabla current**
- **explosión a noches**
- **screening desacoplado pero alimentado por el mismo import**

Y la verdad incómoda es esta:

**el CSV que usas hoy no es el CSV definitivo de Debacu; es un CSV de screening de prueba.**

Está bien para validar la mecánica, pero no para revenue, pickup ni analítica seria.

Si quieres, en el siguiente mensaje te dejo ya el bloque operativo completo con:

- cabecera final cerrada,
- SQL de tablas recomendadas,
- claves únicas,
- y flujo Edge Function `dry-run -> commit -> normalize -> screening -> snapshots -> revenue_daily`.

Fuentes

Perfecto. Vamos a dejarlo ya en modo operativo.

Tu documento v1 ya fija bien la base del producto: un solo CSV, una fila por reserva/estancia, mínimos de reserva, bloque de identificación para screening y bloque recomendado para revenue. También deja claro que el mismo archivo debe servir para screening, revenue, pickup y comparativas históricas.

Lo que falta ahora es cerrarlo como arquitectura SaaS real, no como piloto.

---

## 1) CSV oficial definitivo Debacu Unified v1.0

### Cabecera canónica propuesta

property\_code, reservation\_id, reservation\_line\_id, booking\_date, checkin\_date, checkout\_date, reservation\_status, currency, gross\_revenue, commission\_amount, net\_revenue, rooms, adults, children, channel, segment, room\_type, rate\_plan, first\_name, last\_name, document, email, phone, country, cancelled\_at, company, agency, market\_code, source\_system

### Clasificación de columnas

#### Obligatorias duras

Sin estas, el fichero no entra al pipeline unificado:

- property\_code
- reservation\_id
- booking\_date
- checkin\_date
- checkout\_date
- reservation\_status
- currency
- gross\_revenue
- rooms

#### Obligatorias condicionales para screening

No bloquearía el CSV entero por esto, pero sí marcaría la fila como “sin screening fiable” si falla:

- al menos uno de:
  - document
  - email
  - phone

Y recomendable además:

- first\_name
- last\_name

### Recomendadas fuertes para Revenue Intelligence

No deberían ser obligatorias al principio, pero casi sí para sacar valor de verdad:

- reservation\_line\_id
- commission\_amount
- net\_revenue
- adults
- children
- channel
- segment
- room\_type
- rate\_plan

### Opcionales

- cancelled\_at
  - company
  - agency
  - market\_code
  - source\_system
- 

## 2) Normalización de negocio

El PDF base usa status con valores tipo confirmed / cancelled / no\_show / checked\_out. Eso está bien como idea, pero internamente no conviene dejar textos libres del PMS.

### Enum canónico interno

CONFIRMED  
IN\_HOUSE  
CHECKED\_OUT  
CANCELLED  
NO\_SHOW

## Mapeos típicos

Ejemplos:

- confirmed, booked, ok, active → CONFIRMED
- in\_house, checked\_in, staying → IN\_HOUSE
- checked\_out, departed → CHECKED\_OUT
- cancelled, canceled → CANCELLED
- noshow, no\_show → NO\_SHOW

No dejes que la UI ni las métricas dependan del texto original del PMS.

---

## 3) Reglas de validación reales

El documento base ya marca formato de fechas YYYY-MM-DD, revenue  $\geq 0$  y coherencias básicas entre fechas. Eso se mantiene, pero hay que endurecerlo algo más.

### Reglas duras por fila

#### Fechas

- booking\_date formato YYYY-MM-DD
- checkin\_date formato YYYY-MM-DD
- checkout\_date formato YYYY-MM-DD
- booking\_date  $\leq$  checkin\_date
- checkout\_date  $>$  checkin\_date

No aceptaría checkout\_date = checkin\_date salvo que quieras admitir day-use. Mi consejo: no lo admitas en v1.

#### Importe

- gross\_revenue  $\geq 0$
- si viene commission\_amount, entonces commission\_amount  $\geq 0$
- si viene commission\_amount, entonces commission\_amount  $\leq$  gross\_revenue
- si viene net\_revenue, entonces net\_revenue  $\geq 0$

- si vienen `gross_revenue` y `commission_amount` pero no `net_revenue`, se calcula:
  - `net_revenue = gross_revenue - commission_amount`

#### Habitaciones y ocupación

- `rooms` entero  $\geq 1$
- si viene `adults`, entero  $\geq 0$
- si viene `children`, entero  $\geq 0$

#### Estado

`reservation_status` solo puede ser uno de los valores canónicos, tras mapping.

#### Moneda

- `currency` ISO 4217 de 3 letras
- ejemplo: EUR, USD, GBP

#### Screening

- si faltan `document`, `email` y `phone` a la vez:
    - la fila no se rechaza si sirve para `revenue`
    - pero queda `screening_eligible = false`
- 

## 4) Política de errores de importación

Esto es importante. Si no defines esto, luego el hotel no sabe qué ha pasado.

#### Error fatal de batch

Se rechaza el archivo entero si:

- faltan columnas obligatorias duras
- el separador rompe la estructura
- las cabeceras no se pueden mapear
- más del 20% de filas tienen errores duros

## Error por fila

Se importa el lote y se marca la fila con error si:

- fecha inválida
- revenue inválido
- rooms inválido
- clave natural duplicada dentro del mismo batch

## Warning por fila

Se importa la fila pero con aviso si:

- faltan campos de screening
- falta channel
- falta segment
- falta room\_type
- falta rate\_plan

Eso es mucho más realista que pedir perfección al hotel.

---

## 5) Clave natural correcta

Aquí está uno de los puntos más delicados.

### No usar solo

`org_id + reservation_id`

Eso falla si:

- una reserva tiene varias líneas
- el PMS desglosa por habitación
- una reserva cambia parcialmente
- hay colisiones entre propiedades

### Clave natural recomendada

`org_id + property_code + reservation_id +  
coalesce(reservation_line_id, '1')`

Y con eso generas un hash técnico:

reservation\_key

Ejemplo conceptual:

sha256(org\_id|property\_code|reservation\_id|reservation\_line\_id)

Ese reservation\_key será la base de todo:

- estado actual
  - snapshots
  - nights
  - screening vinculado a la reserva
- 

## 6) Identidad para screening

No mezcles “reserva” con “persona” en una única clave.

### identity\_key

Debe generarse con prioridad:

1. document
2. email
3. phone
4. first\_name + last\_name + country como fallback débil

Normalización recomendada:

- trim
- lowercase para email
- quitar espacios
- quitar guiones en teléfono/documento si procede
- aplicar hash HMAC con pepper

Ejemplo conceptual:

identity\_key = HMAC\_SHA256(normalized\_best\_identifier, pepper)

### identity\_strength

Guarda además:

- HIGH si hay documento
- MEDIUM si hay email o teléfono



- LOW si solo hay nombre+apellido+país

Esto te evita vender como “exacto” algo que no lo es.

---

## 7) Modelo de tablas recomendado

### 7.1 Importaciones

#### debacu\_eval\_import\_batches

```
create table if not exists public.debacu_eval_import_batches (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null,  
  property_code text not null,  
  import_profile_id uuid null,  
  source_file_name text not null,  
  source_file_sha256 text not null,  
  source_system text null,  
  status text not null check (status in (  
    'PENDING',  
    'DRY_RUN_OK',  
    'DRY_RUN_ERROR',  
    'COMMITTED',  
    'PARTIAL_ERROR'  
  )),  
  rows_total int not null default 0,  
  rows_ok int not null default 0,  
  rows_error int not null default 0,  
  rows_warning int not null default 0,  
  uploaded_at timestamptz not null default now(),  
  committed_at timestamptz null  
);  
create index if not exists debacu_eval_import_batches_org_idx  
  on public.debacu_eval_import_batches(org_id, uploaded_at desc);
```

#### debacu\_eval\_import\_rows

```
create table if not exists public.debacu_eval_import_rows (  
  id uuid primary key default gen_random_uuid(),  
  batch_id uuid not null references  
  public.debacu_eval_import_batches(id) on delete cascade,  
  row_number int not null,  
  raw_payload jsonb not null,  
  normalized_payload jsonb null,  
  validation_status text not null check (validation_status in  
  ('OK', 'WARNING', 'ERROR')),
```

```

validation_errors jsonb not null default '[]'::jsonb,
validation_warnings jsonb not null default '[]'::jsonb,
reservation_key text null,
identity_key text null,
created_at timestamptz not null default now(),
unique(batch_id, row_number)
);
create index if not exists debacu_eval_import_rows_batch_idx
on public.debacu_eval_import_rows(batch_id);

```

---

## 7.2 Estado actual de reservas

### debacu\_eval\_reservations

```

create table if not exists public.debacu_eval_reservations (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  property_code text not null,
  reservation_key text not null,
  reservation_id text not null,
  reservation_line_id text null,
  booking_date date not null,
  checkin_date date not null,
  checkout_date date not null,
  reservation_status text not null check (reservation_status in (
    'CONFIRMED', 'IN_HOUSE', 'CHECKED_OUT', 'CANCELLED', 'NO_SHOW'
  )),
  currency text not null,
  gross_revenue numeric(12,2) not null,
  commission_amount numeric(12,2) null,
  net_revenue numeric(12,2) null,
  rooms int not null,
  adults int null,
  children int null,
  channel text null,
  segment text null,
  room_type text null,
  rate_plan text null,
  cancelled_at timestamptz null,
  company text null,
  agency text null,
  market_code text null,
  source_system text null,
  first_seen_batch_id uuid null references
public.debacu_eval_import_batches(id),
  last_seen_batch_id uuid null references
public.debacu_eval_import_batches(id),
  first_seen_at timestamptz not null default now(),

```

```

    last_seen_at timestamptz not null default now(),
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique(org_id, reservation_key)
);
create index if not exists debacu_eval_reservations_org_property_idx
    on public.debacu_eval_reservations(org_id, property_code,
    checkin_date);
create index if not exists debacu_eval_reservations_status_idx
    on public.debacu_eval_reservations(org_id, reservation_status);

```

---

### 7.3 Identidad ligada a reserva

#### debacu\_eval\_reservation\_identities

```

create table if not exists public.debacu_eval_reservation_identities (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    reservation_key text not null,
    identity_key text not null,
    identity_strength text not null check (identity_strength in
('HIGH', 'MEDIUM', 'LOW')),
    first_name_enc text null,
    last_name_enc text null,
    document_hash text null,
    email_hash text null,
    phone_hash text null,
    country text null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique(org_id, reservation_key, identity_key)
);
create index if not exists
debacu_eval_reservation_identities_identity_idx
    on public.debacu_eval_reservation_identities(org_id, identity_key);

```

Aquí puedes guardar PII cifrada si la necesitas para uso operativo y, aparte, hashes para matching.

---

### 7.4 Snapshots históricos para pickup

#### debacu\_eval\_reservation\_snapshots

```

create table if not exists public.debacu_eval_reservation_snapshots (
    id uuid primary key default gen_random_uuid(),
    batch_id uuid not null references
public.debacu_eval_import_batches(id) on delete cascade,

```

```

    org_id uuid not null,
    property_code text not null,
    reservation_key text not null,
    snapshot_date date not null default current_date,
    booking_date date not null,
    checkin_date date not null,
    checkout_date date not null,
    reservation_status text not null,
    currency text not null,
    gross_revenue numeric(12,2) not null,
    commission_amount numeric(12,2) null,
    net_revenue numeric(12,2) null,
    rooms int not null,
    adults int null,
    children int null,
    channel text null,
    segment text null,
    room_type text null,
    rate_plan text null,
    created_at timestamptz not null default now()
);
create index if not exists
debacu_eval_reservation_snapshots_org_checkin_idx
    on public.debacu_eval_reservation_snapshots(org_id, property_code,
checkin_date);
create index if not exists debacu_eval_reservation_snapshots_batch_idx
    on public.debacu_eval_reservation_snapshots(batch_id);
Esta tabla es append-only. No se actualiza. Solo se inserta.

Sin esto, pickup serio no existe.

```

---

## 7.5 Explosión a noches

### debacu\_eval\_stay\_nights

```

create table if not exists public.debacu_eval_stay_nights (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_code text not null,
    reservation_key text not null,
    stay_date date not null,
    reservation_status text not null,
    rooms int not null default 1,
    room_nights numeric(10,2) not null default 1,
    allocated_gross_revenue numeric(12,2) not null default 0,
    allocated_net_revenue numeric(12,2) null,
    channel text null,
    segment text null,

```

```

    room_type text null,
    rate_plan text null,
    source_batch_id uuid null references
public.debacu_eval_import_batches(id),
    created_at timestamptz not null default now(),
    unique(org_id, property_code, reservation_key, stay_date)
);
create index if not exists debacu_eval_stay_nights_daily_idx
on public.debacu_eval_stay_nights(org_id, property_code, stay_date);
Regla de reparto inicial:

```

- número de noches = checkout\_date - checkin\_date
- revenue asignado por noche = gross\_revenue / noches
- room\_nights por noche = rooms

Más adelante ya podrás sofisticarlo si hay paquetes o revenue no alojativo.

---

## 7.6 Agregados para dashboard

### debacu\_eval\_revenue\_daily

```

create table if not exists public.debacu_eval_revenue_daily (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_code text not null,
    metric_date date not null,
    reservations_count int not null default 0,
    room_nights numeric(12,2) not null default 0,
    gross_revenue numeric(14,2) not null default 0,
    net_revenue numeric(14,2) not null default 0,
    cancelled_reservations int not null default 0,
    no_show_reservations int not null default 0,
    adr numeric(12,2) null,
    revpar numeric(12,2) null,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    unique(org_id, property_code, metric_date)
);

```

El revpar requiere que ya tengas en Debacu el inventario de habitaciones de cada propiedad. El PDF base menciona RevPAR y ocupación como objetivo analítico, pero eso solo sale bien si combinas el CSV con metadatos internos de propiedad.

---

## 8) Qué calcula este modelo desde el día 1

Con el CSV unificado bien cargado y las tablas anteriores, puedes sacar:

### Sí, de forma sólida

- ADR
- lead time
- revenue bruto
- revenue neto
- room nights
- cancelaciones
- no-shows
- mix de canal
- mix de segmento
- comparativa mensual
- comparativa diaria
- pickup entre imports
- reservas futuras por fecha de llegada

### Sí, pero depende de tener inventario de habitaciones

- ocupación
- RevPAR

### No sale solo con este CSV

- pricing competitivo real
- benchmark de mercado
- cuota vs comp set
- price position tipo OTA Insight

Eso requiere fuentes externas.

---

## 9) Flujo de ingestión definitivo

### Paso 1. Upload

El hotel sube CSV.

### Paso 2. Dry-run

- detección de separador
- validación de cabecera
- mapping de columnas
- normalización de valores
- preview
- conteo de errores/warnings

### Paso 3. Commit

Se crea `import_batch`.

### Paso 4. Persistencia raw

Se guarda cada fila en `debacu_eval_import_rows`.

### Paso 5. Upsert current

Se hace upsert en `debacu_eval_reservations` por `org_id` + `reservation_key`.

### Paso 6. Snapshot

Se inserta snapshot append-only en `debacu_eval_reservation_snapshots`.

### Paso 7. Identity

Se crea o actualiza `debacu_eval_reservation_identities`.

### Paso 8. Screening

Se ejecuta screening sobre filas `screening_eligible = true`.

### Paso 9. Explosión a noches

Se recalcula `debacu_eval_stay_nights` para las reservas afectadas.

## Paso 10. Agregados

Se recalculan métricas diarias y mensuales.

---

### 10) Dedupe realista

#### Dentro del mismo batch

Si se repite la misma clave natural:

- marcas la segunda fila como error
- no la procesas

#### Entre batches distintos

No lo tratas como duplicado.

Lo tratas como:

- nueva observación del mismo reservation\_key

Y por tanto:

- actualizas debacu\_eval\_reservations
- insertas snapshot nuevo

Eso es exactamente lo que necesitas para pickup.

---

### 11) Reglas de pickup

Para pickup no compares solo revenue actual contra revenue anterior.

Compara por ventana de llegada.

Ejemplos que podrás sacar:

- reservas hechas para check-in en próximos 15 días
- room nights on the books hoy vs hace 7 días
- gross revenue on the books hoy vs hace 30 días
- cancelaciones nuevas desde la carga anterior
- ritmo de entrada de reservas por canal

Eso es mucho más útil para hotel que una estadística aislada.

---



## 12) Cómo encaja con tu screening actual

Tu pipeline actual ya hace:

- upload
- dry-run
- validación
- screening
- run
- resultados
- alertas
- no duplicar innecesariamente
- detectar cambios de riesgo posteriores

Eso no hay que tirarlo. Hay que encapsularlo dentro del pipeline unificado. El documento base ya prevé que el mismo CSV alimente screening y analítica.

### Decisión correcta

Mantén dos perfiles de entrada:

#### screening-lite

Para pilotos y pruebas rápidas.  
Admite el CSV actual, más pobre.

#### debacu-unified-v1

Es el oficial.

Exige el modelo completo de reserva + revenue.

Así no rompes lo que ya funciona, pero no contaminas la arquitectura definitiva.

---

## 13) Recomendación de producto, sin maquillaje

Lo sensato es esto:

- no pedir varios CSV al hotel
- no pedir consulta manual por huésped
- no exigir perfección de datos personales

- sí exigir bloque mínimo de reserva y revenue
- sí guardar snapshots desde el principio
- sí derivar noches internamente
- sí separar raw, current, snapshots, identities y agregados

Lo que **no** conviene hacer:

- meter todo en `guest_stays` y cruzar los dedos
  - basar pickup solo en el último estado actual
  - usar `reservation_id` pelado como clave única global
  - rechazar lotes completos por faltas menores de screening
  - pretender RevPAR serio sin inventario de habitaciones por propiedad
- 

## 14) Qué haría yo ahora mismo como siguiente paso técnico

Orden correcto:

1. cerrar la cabecera oficial `Debacu Unified v1.0`
  2. crear tablas de import, reservations, identities, snapshots y stay\_nights
  3. adaptar la Edge Function actual para soportar:
    - `mode = dry_run`
    - `mode = commit`
    - `profile = screening_lite | debacu_unified_v1`
  4. mantener screening actual
  5. añadir explosión a noches
  6. construir vistas/materializados para:
    - revenue daily
    - pickup
    - channel mix
    - monthly comparison
-

## 15) Cabecera final que yo daría ya como oficial

property\_code, reservation\_id, reservation\_line\_id, booking\_date, checkin\_date, checkout\_date, reservation\_status, currency, gross\_revenue, commission\_amount, net\_revenue, rooms, adults, children, channel, segment, room\_type, rate\_plan, first\_name, last\_name, document, email, phone, country, cancelled\_at, company, agency, market\_code, source\_system

Y si quieres una versión mínima oficial para hoteles con peor PMS:

property\_code, reservation\_id, booking\_date, checkin\_date, checkout\_date, reservation\_status, currency, gross\_revenue, rooms, first\_name, last\_name, document, email, phone, country

Pero dejando claro que esa versión mínima:

- sirve para screening
  - sirve para analítica básica
  - no sirve igual de bien para revenue avanzado
- 

## 16) Conclusión cerrada

La arquitectura correcta ya no es una idea. Es esta:

- **1 CSV canónico**
- **mapeo por PMS**
- **raw import**
- **estado actual de reservas**
- **snapshots históricos**
- **identidades separadas**
- **explosión a noches**
- **agregados diarios/mensuales**
- **screening acoplado al mismo pipeline**

Eso sí escala.

En el siguiente paso te dejo directamente el bloque técnico siguiente, ya listo para copiar:

- SQL completo de creación de tablas
- propuesta de índices
- y pseudocódigo/estructura de la Edge Function `csv_unified_import` con `dry_run` y `commit`.

Bien. Entonces **no ejecutes el SQL que te pasé antes tal cual** para 1.1 y 1.2.

Lo que muestras deja una cosa clara:

- `debacu_eval_import_batches` ya **está en producción y funcionando** para screening.
- `debacu_eval_import_rows` **también existe**, pero su estructura actual está pensada para screening/matching, no para import unificado de reservas.

La conclusión práctica es esta:

### No conviene romper esas dos tablas ahora

Porque hoy ya te están soportando un flujo real:

- batch
- rows
- screening
- resultados

Y si las modificas agresivamente ahora, puedes cargarte el pipeline que ya funciona.

---

### Qué haría yo

#### Opción correcta: compatibilidad, no sustitución

Mantener:

- `debacu_eval_import_batches` como tabla legacy/operativa de import actual
- `debacu_eval_import_rows` como tabla legacy/operativa de screening actual

Y crear **nuevo bloque unificado** separado para Revenue + CSV oficial.

Eso es mucho más sensato que intentar reciclar ahora `import_rows`, porque tu `import_rows` actual tiene campos como:

- `identity_key`
- `full_name`
- `checkin_date`
- `checkout_date`
- `match_status`
- `risk_band_at_import`

- `error_code`

Eso no es una tabla raw genérica de import.

Eso ya es prácticamente una tabla de procesamiento de screening.

---

## Decisión recomendada

### Mantener tablas existentes

- `debacu_eval_import_batches`
- `debacu_eval_import_rows`

### Crear nuevas tablas para el CSV unificado

Con nombres nuevos, para no contaminar ni romper nada:

- `debacu_eval_unified_import_batches`
  - `debacu_eval_unified_import_rows`
  - `debacu_eval_reservations`
  - `debacu_eval_reservation_identities`
  - `debacu_eval_reservation_snapshots`
  - `debacu_eval_stay_nights`
  - `debacu_eval_revenue_daily`
- 

## Por qué esta opción es la buena

Porque ahora mismo tienes dos realidades distintas:

### Realidad 1: screening actual

Tu batch actual tiene esta pinta:

- `file_name`
- `import_type`
- `total_rows`
- `processed_rows`
- `error_rows`

y se usa para archivos tipo FUTURE\_BOOKINGS.

Eso está bien para el flujo actual.

## Realidad 2: import unificado futuro

Necesita cosas nuevas como:

- `property_code`
- `source_file_sha256`
- `separator`
- `status`
- `rows_warning`
- `committed_at`
- `metadata`
- `normalized_payload`
- validación por fila
- snapshots históricos

Eso ya es otro sistema. Intentar meterlo todo dentro de las tablas actuales solo porque el nombre se parece sería mala idea.

---

## Lo que te recomiendo hacer ya

### Fase 1

No tocar `debacu_eval_import_batches` ni `debacu_eval_import_rows`.

### Fase 2

Crear el nuevo bloque unificado en paralelo.

Así:

- el screening actual sigue funcionando
  - puedes desarrollar revenue sin romper nada
  - más adelante decides si fusionas o no
-

## SQL correcto para tu caso actual

### 1) Crear nuevas tablas unificadas, sin tocar las existentes

#### debacu\_eval\_unified\_import\_batches

```
create table if not exists public.debacu_eval_unified_import_batches (  
  id uuid primary key default gen_random_uuid(),  
  org_id uuid not null,  
  property_code text not null,  
  import_profile_code text not null default 'DEBACU_UNIFIED_V1',  
  source_file_name text not null,  
  source_file_sha256 text not null,  
  source_system text null,  
  separator text not null default ',',  
  status text not null check (  
    status in (  
      'PENDING',  
      'DRY_RUN_OK',  
      'DRY_RUN_ERROR',  
      'COMMITTED',  
      'PARTIAL_ERROR'  
    )  
  ),  
  rows_total int not null default 0,  
  rows_ok int not null default 0,  
  rows_warning int not null default 0,  
  rows_error int not null default 0,  
  uploaded_by uuid null,  
  uploaded_at timestamptz not null default now(),  
  committed_at timestamptz null,  
  metadata jsonb not null default '{} '::jsonb  
);  
create index if not exists  
debacu_eval_unified_import_batches_org_uploaded_idx  
  on public.debacu_eval_unified_import_batches (org_id, uploaded_at  
desc);  
create unique index if not exists  
debacu_eval_unified_import_batches_org_filehash_uq  
  on public.debacu_eval_unified_import_batches (org_id, property_code,  
source_file_sha256);
```

---

#### debacu\_eval\_unified\_import\_rows

```
create table if not exists public.debacu_eval_unified_import_rows (  
  id uuid primary key default gen_random_uuid(),
```

```

    batch_id uuid not null references
public.debacu_eval_unified_import_batches(id) on delete cascade,
    org_id uuid not null,
    row_number int not null,
    raw_payload jsonb not null,
    normalized_payload jsonb null,
    validation_status text not null check (
        validation_status in ('OK', 'WARNING', 'ERROR')
    ),
    validation_errors jsonb not null default '[]'::jsonb,
    validation_warnings jsonb not null default '[]'::jsonb,
    reservation_key text null,
    identity_key text null,
    screening_eligible boolean not null default false,
    created_at timestamptz not null default now(),
    unique (batch_id, row_number)
);
create index if not exists debacu_eval_unified_import_rows_batch_idx
    on public.debacu_eval_unified_import_rows (batch_id);
create index if not exists
    debacu_eval_unified_import_rows_batch_status_idx
    on public.debacu_eval_unified_import_rows (batch_id,
validation_status);
create index if not exists
    debacu_eval_unified_import_rows_org_reservation_idx
    on public.debacu_eval_unified_import_rows (org_id, reservation_key);

```

---

## 2) Crear tablas de negocio nuevas

### debacu\_eval\_reservations

```

create table if not exists public.debacu_eval_reservations (
    id uuid primary key default gen_random_uuid(),
    org_id uuid not null,
    property_code text not null,
    reservation_key text not null,
    reservation_id text not null,
    reservation_line_id text null,
    booking_date date not null,
    checkin_date date not null,
    checkout_date date not null,
    reservation_status text not null check (
        reservation_status in ('CONFIRMED', 'IN_HOUSE', 'CHECKED_OUT',
'CANCELLED', 'NO_SHOW')
    ),
    currency text not null,
    gross_revenue numeric(12,2) not null,
    commission_amount numeric(12,2) null,

```



```

net_revenue numeric(12,2) null,
rooms int not null,
adults int null,
children int null,
channel text null,
segment text null,
room_type text null,
rate_plan text null,
cancelled_at timestamptz null,
company text null,
agency text null,
market_code text null,
source_system text null,
first_seen_batch_id uuid null references
public.debacu_eval_unified_import_batches(id),
last_seen_batch_id uuid null references
public.debacu_eval_unified_import_batches(id),
first_seen_at timestamptz not null default now(),
last_seen_at timestamptz not null default now(),
created_at timestamptz not null default now(),
updated_at timestamptz not null default now(),
unique (org_id, reservation_key)
);
create index if not exists
debacu_eval_reservations_org_property_checkin_idx
on public.debacu_eval_reservations (org_id, property_code,
checkin_date);
create index if not exists debacu_eval_reservations_org_status_idx
on public.debacu_eval_reservations (org_id, reservation_status);

```

---

### debacu\_eval\_reservation\_identities

```

create table if not exists public.debacu_eval_reservation_identities (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  reservation_key text not null,
  identity_key text not null,
  identity_strength text not null check (
    identity_strength in ('HIGH', 'MEDIUM', 'LOW')
  ),
  first_name_enc text null,
  last_name_enc text null,
  document_hash text null,
  email_hash text null,
  phone_hash text null,
  country text null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),

```

```
    unique (org_id, reservation_key, identity_key)
);
create index if not exists
debacu_eval_reservation_identities_org_identity_idx
on public.debacu_eval_reservation_identities (org_id, identity_key);
```

---

### debacu\_eval\_reservation\_snapshots

```
create table if not exists public.debacu_eval_reservation_snapshots (
    id uuid primary key default gen_random_uuid(),
    batch_id uuid not null references
public.debacu_eval_unified_import_batches(id) on delete cascade,
    org_id uuid not null,
    property_code text not null,
    reservation_key text not null,
    snapshot_date date not null default current_date,
    booking_date date not null,
    checkin_date date not null,
    checkout_date date not null,
    reservation_status text not null check (
        reservation_status in ('CONFIRMED', 'IN_HOUSE', 'CHECKED_OUT',
'CANCELLED', 'NO_SHOW')
    ),
    currency text not null,
    gross_revenue numeric(12,2) not null,
    commission_amount numeric(12,2) null,
    net_revenue numeric(12,2) null,
    rooms int not null,
    adults int null,
    children int null,
    channel text null,
    segment text null,
    room_type text null,
    rate_plan text null,
    cancelled_at timestamptz null,
    created_at timestamptz not null default now()
);
create index if not exists
debacu_eval_reservation_snapshots_org_property_checkin_idx
on public.debacu_eval_reservation_snapshots (org_id, property_code,
checkin_date);
create index if not exists
debacu_eval_reservation_snapshots_org_snapshot_idx
on public.debacu_eval_reservation_snapshots (org_id, snapshot_date);
```

---

### debacu\_eval\_stay\_nights

```

create table if not exists public.debacu_eval_stay_nights (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  property_code text not null,
  reservation_key text not null,
  stay_date date not null,
  reservation_status text not null check (
    reservation_status in ('CONFIRMED', 'IN_HOUSE', 'CHECKED_OUT',
'CANCELLED', 'NO_SHOW')
  ),
  rooms int not null default 1,
  room_nights numeric(12,2) not null default 1,
  allocated_gross_revenue numeric(12,2) not null default 0,
  allocated_net_revenue numeric(12,2) null,
  channel text null,
  segment text null,
  room_type text null,
  rate_plan text null,
  source_batch_id uuid null references
public.debacu_eval_unified_import_batches(id),
  created_at timestamptz not null default now(),
  unique (org_id, property_code, reservation_key, stay_date)
);
create index if not exists debacu_eval_stay_nights_org_property_stay_idx
  on public.debacu_eval_stay_nights (org_id, property_code, stay_date);

```

---

### debacu\_eval\_revenue\_daily

```

create table if not exists public.debacu_eval_revenue_daily (
  id uuid primary key default gen_random_uuid(),
  org_id uuid not null,
  property_code text not null,
  metric_date date not null,
  reservations_count int not null default 0,
  room_nights numeric(12,2) not null default 0,
  gross_revenue numeric(14,2) not null default 0,
  net_revenue numeric(14,2) not null default 0,
  cancelled_reservations int not null default 0,
  no_show_reservations int not null default 0,
  adr numeric(12,2) null,
  revpar numeric(12,2) null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (org_id, property_code, metric_date)
);
create index if not exists
debacu_eval_revenue_daily_org_property_date_idx

```

```
on public.debacu_eval_revenue_daily (org_id, property_code,  
metric_date);
```

---

## Importante: no intentes “renombrar” las tablas actuales

No hagas ahora cosas como:

- renombrar debacu\_eval\_import\_batches
- meterle 8 columnas nuevas de golpe
- cambiar debacu\_eval\_import\_rows para que pase de screening a raw genérico

Eso sería precipitado.

Ahora mismo lo limpio es trabajar en paralelo.

---

## Qué significa esto para tu arquitectura

### Lo actual queda así

- debacu\_eval\_import\_batches → screening actual
- debacu\_eval\_import\_rows → screening actual

### Lo nuevo queda así

- debacu\_eval\_unified\_import\_batches → ingestión unificada
  - debacu\_eval\_unified\_import\_rows → validación raw/normalizada
  - resto de tablas → revenue + snapshots + nights + métricas
- 

## Ventaja real

Podrás tener durante un tiempo:

### pipeline legacy

CSV screening -> import\_batches -> import\_rows ->  
screening\_results

y en paralelo:

## pipeline nuevo

CSV unificado -> unified\_import\_batches -> unified\_import\_rows -> reservations -> snapshots -> stay\_nights -> revenue\_daily

Eso te permite evolucionar sin parar producción.

---

## Mi recomendación directa

Haz esto en este orden:

### Paso 1

Crear **solo** estas tablas nuevas:

- debacu\_eval\_unified\_import\_batches
- debacu\_eval\_unified\_import\_rows
- debacu\_eval\_reservations
- debacu\_eval\_reservation\_identities
- debacu\_eval\_reservation\_snapshots
- debacu\_eval\_stay\_nights
- debacu\_eval\_revenue\_daily

### Paso 2

No tocar nada del screening actual.

### Paso 3

Montar una nueva Edge Function para el import unificado.

### Paso 4

Más adelante, cuando ya funcione, decides si:

- mantienes ambos pipelines
  - o fusionas parte de la lógica
-

## Conclusión clara

Con lo que enseñas, la decisión correcta no es migrar las tablas existentes.

La decisión correcta es:

**mantener el screening actual intacto y crear el sistema unificado al lado.**

Es menos elegante sobre el papel, pero mucho más seguro y profesional en una base que ya tiene datos reales.

El siguiente paso lógico es darte el **SQL completo con triggers de updated\_at y las funciones RPC para rebuild\_stay\_nights y refresh\_revenue\_daily**, ya adaptadas para usar `debacu_eval_unified_import_batches` en lugar de las tablas legacy.

Perfecto. Entonces **no montamos RPCs**.

Eso cambia el cómo, pero no el modelo.

La decisión queda así:

## Regla de arquitectura

- **Base de datos:** tablas, índices, constraints, triggers simples si hacen falta
- **Lógica de negocio: solo Edge Functions**
- **Nada de RPC** para rebuilds, refreshes o procesamiento del CSV

Y, sinceramente, para vuestro caso tiene sentido.

Con Edge Functions controlas mejor:

- validación
- trazabilidad
- logs
- reintentos
- seguridad con service role
- orquestación por fases
- versionado de lógica

Para un pipeline de ingestión y screening, es mejor enfoque que repartir lógica entre SQL y RPC.

---

## Qué mantenemos igual

Seguimos con:

- `debacu_eval_unified_import_batches`
- `debacu_eval_unified_import_rows`

y reutilizamos las tablas que ya tienes bien montadas:

- `debacu_eval_reservations`
- `debacu_eval_reservation_identities`
- `debacu_eval_reservation_snapshots`
- `debacu_eval_stay_nights`

**Y de momento no tocamos:**

- `debacu_eval_revenue_daily`
- `debacu_eval_unified_revenue_daily`

hasta decidir cómo conectarlo con el módulo revenue existente.

---

## Qué quitamos del plan anterior

Eliminamos por completo estas ideas:

- `debacu_eval_rebuild_stay_nights(...)`
- `debacu_eval_refresh_revenue_daily(...)`

No se crean RPCs.

---

## Cómo queda el flujo correcto con Edge Functions

### Edge Function principal

Nombre razonable:

`debacu_eval_csv_unified_import`

### Modos

- `dry_run`
- `commit`

## Perfil

- DEBACU\_UNIFIED\_V1
  - opcionalmente más adelante SCREENING\_LITE
- 

## Flujo dry\_run

La función:

1. recibe CSV
2. detecta separador
3. parsea cabecera
4. normaliza alias
5. valida fila a fila
6. genera preview
7. devuelve:
  - filas OK
  - warnings
  - errores
  - resumen
  - columnas detectadas

Aquí **no escribe en tablas finales**.

Opcional:

- puede crear batch técnico temporal
  - pero yo al principio ni eso haría
- 

## Flujo commit

La función hace esto, en este orden:

### 1. Resolver propiedad

Convierte:



- `property_code` CSV  
a
- propiedad válida del org

Pero como tus tablas actuales de reservas usan `property_code`, puedes seguir guardando `property_code` ahí sin problema.

## 2. Crear batch

Inserta en:

- `debacu_eval_unified_import_batches`

## 3. Guardar filas raw/normalizadas

Inserta en:

- `debacu_eval_unified_import_rows`

## 4. Upsert reservas actuales

Inserta o actualiza en:

- `debacu_eval_reservations`

clave:

- `org_id`
- `reservation_key`

## 5. Insertar snapshot append-only

Inserta en:

- `debacu_eval_reservation_snapshots`

una fila por reserva procesada en ese batch

## 6. Upsert identidades

Inserta o actualiza en:

- `debacu_eval_reservation_identities`

## 7. Ejecutar screening

Solo sobre filas con:

- `screening_eligible = true`

Y aquí reutilizas tu lógica actual de screening, sin rehacer todo.

## 8. Rebuild stay nights

La propia Edge Function:

- borra las noches anteriores de la reserva
- genera nuevas noches
- inserta en `debaeu_eval_stay_nights`

Todo desde TypeScript, no SQL procedural.

## 9. Marcar batch final

Actualiza:

- `rows_ok`
  - `rows_warning`
  - `rows_error`
  - `status`
  - `committed_at`
- 

## Ventaja de hacerlo así

La lógica queda toda en un único sitio o en helpers TS:

- más fácil de depurar
- más fácil de testear
- más fácil de versionar
- no dependes de RPCs ocultas en SQL
- puedes trocear lotes grandes y hacer reintentos

Para este caso, es mejor.

---

## Qué SQL sí tiene sentido entonces

Solo el estructural:

## Sí

- tablas
- índices
- unique constraints
- foreign keys
- check constraints básicas
- triggers updated\_at si quieres

## No

- RPCs
  - lógica de rebuild
  - lógica de refresh
  - lógica de negocio compleja
- 

## Qué haría yo ahora

### Paso 1

Crear solo estas tablas si aún no están:

- debacu\_eval\_unified\_import\_batches
- debacu\_eval\_unified\_import\_rows

### Paso 2

No crear ninguna RPC.

### Paso 3

Implementar Edge Function con helpers internos:

- parseCsv
- normalizeHeaders
- normalizeStatus
- buildReservationKey
- buildIdentityKey

- `validateUnifiedRow`
  - `upsertReservations`
  - `insertSnapshots`
  - `upsertIdentities`
  - `rebuildStayNightsForReservation`
  - `runScreeningForEligibleRows`
- 

## Cómo debería hacerse `rebuildStayNights` dentro de la Edge Function

Sin RPC, la lógica es esta:

### por cada reserva afectada

1. calcular `noches = checkout_date - checkin_date`
2. borrar de `debacu_eval_stay_nights`:
  - `org_id`
  - `property_code`
  - `reservation_key`
3. repartir revenue por noche
4. insertar una fila por cada noche desde `checkin` hasta `checkout - 1`

Eso se hace perfectamente desde la Edge Function con Supabase client usando service role.

---

## Lo único delicado

Si el lote es muy grande, no conviene hacer:

- un `insert` por noche, por reserva, uno a uno, sin batching

Ahí hay que trabajar bien.

### Estrategia práctica

- construir arrays en memoria
- hacer inserts por bloques, por ejemplo de 500 o 1000 filas

- igual para snapshots
- igual para unified\_import\_rows

Eso sí es importante.

---

## Qué haría con `revenue_daily`

Ahora mismo, nada.

Primero cerraría:

- ingestión
- reservations
- snapshots
- identities
- stay\_nights
- screening

Y después montaría otra Edge Function separada, por ejemplo:

- `debacu_eval_revenue_materialize_daily`

que lea de `debacu_eval_stay_nights` y alimente:

- `debacu_eval_revenue_daily`  
o
- `debacu_eval_unified_revenue_daily`

Pero no mezclaría eso en la primera función.

Si lo metes todo a la vez, la lías y te costará depurar.

---

## Decisión final recomendada

### Edge Function 1

`debacu_eval_csv_unified_import`

Hace:

- dry-run
- commit
- reservations

- snapshots
- identities
- stay\_nights
- screening

## Edge Function 2

debacu\_eval\_revenue\_rebuild\_daily

Hace:

- agregación diaria desde stay\_nights
- opcionalmente pickup/materializados

Separado. Mucho mejor.

---

## Conclusión

Bien hecho parar aquí, porque meter RPCs si vuestra arquitectura ya va por Edge Functions solo habría creado duplicidad y mantenimiento innecesario.

La arquitectura correcta para vosotros queda así:

- **SQL** para estructura
- **Edge Functions** para toda la lógica
- **sin RPC**

El siguiente paso correcto ya no es SQL.

Es darte la **estructura completa del fichero index.ts de la Edge Function debacu\_eval\_csv\_unified\_import**, con helpers y flujo dry\_run / commit, adaptado a las tablas que realmente tienes.

|  |
|--|
|  |
|  |

Y ahemos creado create or replace function public.set\_updated\_at()

returns trigger

language plpgsql

as \$\$

```
begin
```

```
    new.updated_at = now();
```

```
    return new;
```

```
end;
```

```
$$;
```

```
drop trigger if exists trg_debacu_eval_reservations_updated_at on  
public.debacu_eval_reservations;
```

```
create trigger trg_debacu_eval_reservations_updated_at
```

```
before update on public.debacu_eval_reservations
```

```
for each row execute function public.set_updated_at();
```

```
drop trigger if exists trg_debacu_eval_reservation_identities_updated_at on  
public.debacu_eval_reservation_identities;
```

```
create trigger trg_debacu_eval_reservation_identities_updated_at
```

```
before update on public.debacu_eval_reservation_identities
```

```
for each row execute function public.set_updated_at();
```

```
create table if not exists public.debacu_eval_unified_import_rows (
```

```
    id uuid primary key default gen_random_uuid(),
```

```
    batch_id uuid not null references public.debacu_eval_unified_import_batches(id) on delete  
cascade,
```

```
    org_id uuid not null,
```

```
    property_code text not null,
```

```
    row_number int not null,
```

```
    raw_payload jsonb not null,
```

```
    normalized_payload jsonb null,
```

```
validation_status text not null check (  
    validation_status in ('OK', 'WARNING', 'ERROR')  
),  
validation_errors jsonb not null default '[]'::jsonb,  
validation_warnings jsonb not null default '[]'::jsonb,  
reservation_key text null,  
identity_key text null,  
screening_eligible boolean not null default false,  
created_at timestamptz not null default now(),  
unique (batch_id, row_number)  
);
```

```
create index if not exists debacu_eval_unified_import_rows_batch_idx  
on public.debacu_eval_unified_import_rows (batch_id);
```

```
create index if not exists debacu_eval_unified_import_rows_batch_status_idx  
on public.debacu_eval_unified_import_rows (batch_id, validation_status);
```

```
create index if not exists debacu_eval_unified_import_rows_org_property_reservation_idx  
on public.debacu_eval_unified_import_rows (org_id, property_code, reservation_key);
```